

Competitive Programming

Section 7: Foundation mixed contest

Competitive Programming Course

2026-07-28

Table of contents

1	Why a mixed contest now	3
2	Complete local contest set	4
3	Contest triage	5
4	A diagnosis routine, not an answer key	6
5	Reading tie-breaking rules	7
6	Upsolving workflow	8
7	Common mixed-contest mistakes	9
8	Before you submit	10
9	Exercises	11

1 Why a mixed contest now

Sections 1 through 6 introduced the foundation tools:

- constraints and I/O;
- simulation and brute force;
- counting and frequency maps;
- prefix sums and difference arrays;
- sorting;
- two pointers and sliding windows.

Real contests do not label problems with the technique. A mixed section trains the missing skill: deciding which tool the statement is asking for.

This section should be attempted like a short virtual contest. Read all five problems first, estimate the intended complexity, solve the easiest one, and then upsolve carefully.

2 Complete local contest set

Problem	Title
A	Inventory Cleanup
B	Peak Load
C	Best Study Streak
D	Balanced Break
E	Circular Patrol

Problems D and E extend the original three-problem set. Include them in the same triage pass: D adds another candidate-enumeration exercise, while E makes explicit state simulation part of the mixed contest.

3 Contest triage

When you open a set, do a quick pass:

```
for each problem:  
    read statement and constraints  
    identify input size and value range  
    guess one or two possible tools  
    mark confidence: high / medium / low
```

Do not spend 25 minutes on the first problem just because it is first. A contest is a scheduling problem too.

4 A diagnosis routine, not an answer key

Do not read the local problem names as technique labels. For each statement, write down observations before choosing an algorithm:

1. What changes between operations or positions?
2. What is the largest input dimension?
3. What direct approach would be too slow, and why?
4. Does the answer depend on values, equality classes, order, or a contiguous range?
5. Which earlier tool has an invariant that could rule out the slow approach?

At this stage, keep two candidate ideas if necessary. Check each candidate against the constraints and against one deliberately awkward small example. The goal is to eliminate wrong models yourself, not to recognize a keyword and immediately reach for a memorized template.

For this mixed section, attempt Problems A through E before continuing. Their statements contain enough information to diagnose them using Sections 1–6. Return to this lesson after your attempt to review the contest process; use the editorial only for a full upsolve.

5 Reading tie-breaking rules

Tie-breaking rules are part of the problem, not output decoration.

For each answer variable, ask:

```
What is the primary objective?  
What is the secondary objective?  
What should happen if everything ties?
```

Write the comparison rule explicitly before coding. For example, if the first objective is a maximum and the second is an earliest position, decide whether an equal primary value should replace the stored answer or preserve it. This prevents accidental tie behavior caused by $>$ versus $>=$.

6 Upsolving workflow

After attempting the section:

1. Reproduce the failure locally.
2. Identify whether the issue is technique choice, implementation, or edge case.
3. Write or inspect a slow oracle for small cases.
4. Add one targeted test that would have caught the bug.
5. Only then read the full editorial solution.

The local random generators in this course intentionally use slow oracles for small cases. This is how many of the hand-written sample mistakes were caught while building the course.

7 Common mixed-contest mistakes

- Choosing a familiar tool before checking whether its assumptions hold.
- Ignoring a tie-breaker because the primary objective looks familiar.
- Accepting a complexity estimate without multiplying all nested loops.
- Testing only samples instead of an adversarial small case.
- Solving the first problem slowly while easier points are available later.
- Trusting hand-written samples without an oracle.

8 Before you submit

For each problem, write one sentence:

```
The controlling constraint is ...  
The intended complexity is ...  
The key invariant is ...  
The edge case I tested is ...
```

If you cannot fill these in, the solution is not ready.

9 Exercises

Attempt these as a timed set:

1. a_inventory_cleanup
2. b_peak_load
3. c_best_study_streak
4. d_balanced_break
5. e_circular_patrol

Run:

```
python3 sections/07_foundation_mixed_contest/check.py
```